

UNIVERSIDAD PRIVADA DEL NORTE

FACULTAD DE INGENIERÍA

Ingeniería de Sistemas Computacionales

Curso: BASE DATOS AVANZADO Y BIG DATA - CIIN1021P

Docente: Jorge Luis Muñoz Diaz

Caso de Estudio: DataSalud Perú: Diseño e implementación de un sistema integrado de base de datos segura, automatizada e inteligente para la gestión del ciclo de vida de los datos del MINSA

Integrantes:

-José David Jesús Encarnación (N00562634)

-Anderson Torres Gonzales (N00562525)

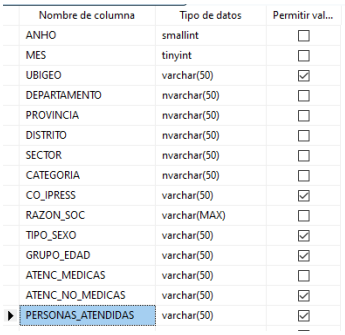
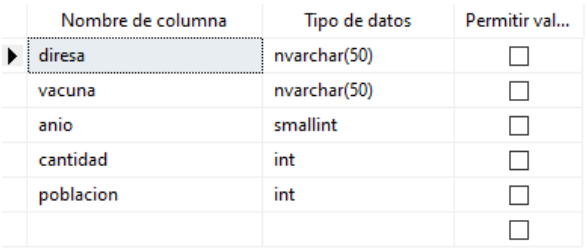
-Luis Alexander Saavedra Povis (N00562712)

-Dhan Janios Trujillo Cardenas (N00287721)

LIMA - PERU | 2026-II

1. Relevamiento, supuestos y articulación curricular

1.1 Selección y caracterización de los datasets

Campo de la ficha	Dataset 1: ConsultaB1_2026_v15.csv	Dataset 2: Trama_esquema_regular_descarga.csv
Fuente de origen	http://datos.susalud.gob.pe/dataset/consulta-b1-produccion-asistencial-en-consulta-ambulatoria	https://files.minsa.gob.pe/s/oegRgjiz54MLfHm?dir=/&editing=false&openfile=true
Tipo de dato	Estructurado	Estructurado
Volumen	100 MB	228 KB
Cantidad de registros	1,048,575 filas	5,597 filas
Formato de archivo	.CSV	.CSV
Esquema		
Licencia de uso	CC0: Dominio público	CC0: Dominio público

1.2 Problema de calidad detectados

Problema	Campo	Consulta de verificación	Registros afectados
Códigos NE_0001 y NE_0002 en lugar de valor real	ConsultaB1.csv: TIPO_SEXO, GRUPO_EDAD, ATENC_MEDICAS, ATENC_NO_MEDICAS, PERSONAS_ATENDIDAS	SELECT …… from …… WHERE …… = 'NE_0001' OR …… = 'NE_0002' SELECT ……, COUNT(*) FROM ConsultaB1 WHERE …… LIKE ‘NE%’ GROUP BY ……	NE_0001 = 30916 registros afectados NE_0002 = 17340 registros afectados
Población igual a 0	Trama_esquema_regular_descarga.csv: poblacion	SELECT COUNT(*) FROM [Trama_esquema_regular_descarga (1)] WHERE poblacion = 0	87 registros afectados
Nombres de DIRESA sin normalizar(varias DIRIS de Lima)	Trama_esquema_regular_descarga.csv: diresa	SELECT DISTINCT diresa FROM [Trama_esquema_regular_descarga (1)]	772 registros afectados

1.3 Requisitos Funcionales

ID	Requerimiento Funcional	Criterio de aceptación
RF-01	El sistema debe cargar la base de atenciones (ConsultaB1) con sus 15 columnas: ANHO, MES, UBIGEO, DEPARTAMENTO, PROVINCIA, DISTRITO, SECTOR, CATEGORIA, CO_IPRESS, RAZON_SOCIAL, TIPO_SEXO, GRUPO_EDAD, ATENC_MEDICAS,	La cantidad de registros cargados es igual a la cantidad de filas del archivo de origen.

	ATENC_NO_MEDICAS y PERSONAS_ATENDIDAS.	
RF-02	El sistema debe cargar la base de vacunación (Trama_esquema_regular_descarga) con las columnas de Diresa, vacuna, año, cantidad y poblacion.	La cantidad de registros cargados es igual a la del archivo de origen, y los totales de cantidad y población coinciden con los del origen
RF-03	El sistema debe identificar los códigos NE_0001 y NE_0002 sin la necesidad de convertirlo en cero o NULL	Ningún valor NE_0001/2 queda cargado como 0 ni como texto en columnas numéricas. El valor numérico queda en NULL, se conserva el código original y se distingue entre NE_0001 y NE_0002

1.4 Supuestos adoptados ante datos incompletos o ambiguos

Supuesto	Justificación técnica	Efecto en el diseño
NE_0001 y NE_0002 significa que la actividad no se realizó en esta IPRESS y que la actividad en esa IPRESS no se realizó en dicho periodo	Es lo que significa según la SUSALUD	En el momento de integración de datos se separan los valores numéricos reales y se registra con un indicador de “actividad no realizada”, siendo que no se cargan como ceros o con una distorsión en los promedios o sumas. Se conserva el código para poder diferenciar en los dos distintos casos: No realizada en la IPRESS y no realizada en dicho periodo.

TIPO_SEXO contiene números los cuales son 1 y 2, donde 1 = masculino y 2 = femenino	Vienen como códigos numéricos sin etiquetas	Se conserva el código de modo que los reportes sean legibles
La columna “vacuna” mezcla el nombre de la vacuna y dosis o refuerzo en un solo texto (por ejemplo: “PENTA 1° DOSIS”), y poblacion puede venir con un valor de 0	En la tabla, vacuna combina el tipo de vacuna y la dosis en un solo texto y hay algunos registros con valor 0.	Se separa vacuna en dos atributos las cuales son tipo de vacuna y dosis o refuerzo, permitiendo comparar la misma vacuna entre dosis. Los registros que poseen poblacion con un número 0 queda en NULL.

1.5 Articulación curricular

Curso	Cómo se retoma en el proyecto	Extensión que se debe aplicar
Base de datos: Consultas SQL y agregaciones	Perfilado de calidad, conteo, consultas críticas y verificación de resultados	Las consultas se usan como evidencias medibles
Ciencia de datos (Curso posterior)	Limpieza, agregación, exploración y construcción de indicadores	Desarrolla competencia para preparar datos para análisis escalable
Ingeniería de software (Curso posterior)	Requisitos verificables, trazabilidad y documentación	Desarrolla competencia para integrar componentes y mantener un sistema reproducible

2. Automatización mediante objetos de base de datos

2.1

Hitos	Fecha	Bloque	Entregable verificable	Responsable	Dependencia
H1: Descargar y Perfilar dataset 1 y 2	17/10/2026	Base	Archivos originales, ficha fuente y perfil de campos	-Anderson Torres Gonzales -José David Jesús Encarnación	Ninguna
H2: Identificar problemas de calidad requisitos y supuestos	18/10/2026	Relevamiento	Tabla con ≥ 3 problemas y conteos, ≥ 3 de Requisitos funcionales y ≥ 3 supuestos	-Luis Alexander Saavedra Povis -Dhan Janios Trujillo Cardenas	H1 terminada
H3: Importar como tablas planas los dataset en archivo .csv	18/10/2026	Base de datos	Tabla de preparación para atenciones y vacunación, registros importados y reporte de filas cargadas y observadas	-Anderson Torres Gonzales	H1 Y H2 terminada
H4: Procedimientos, triggers y funciones	24/10/2026	Automatización	Scrips probados y subido a GitHub	-Dhan Janios Trujillo Cardenas -Luis Alexander Saavedra Povis	H1, H2 y H3 terminada

H5: Implementación de seguridad y control de accesos	31/10/2026	Seguridad	Scrips de creación de roles (administración, analista y auditor), asignación de permisos.	-José David Jesús Encarnación -Anderson Torres Gonzales	H1, H2, H3 y H4 terminada

2.2 Ruta crítica

Dependencia	Por qué es crítica	Qué pasa si se retrasa
H1 a H2	Es necesario descargar y revisar los contenidos de la dataset para que se pueda identificar los campos, estructuras, valores especiales o nulos, y posibles duplicados	Se retrasa la identificación de los problemas de calidad, la elaboración de los requerimientos funcionales y la declaración de supuestos del proyecto
H2 a H3	Antes de importar los datasets al SQL Server, se debe de conocer los tipos de datos, formatos y problemas identificados para definir correctamente las tablas de preparación y evitar errores durante la importación.	La importación puede generar errores, pérdida de información o conversiones incorrectas de valores como NE_0001 y NE_0002
H3 a H4	Los procedimientos almacenados, triggers y funciones necesitan las tablas creadas en SQL Server para poder automatizar, validar registros y comprobar su funcionamiento	No se podrán desarrollar ni probar correctamente los objetos de automatización, retrasando el bloque de procesamiento de datos

H4 a H5	Es necesario conocer los procedimientos almacenados, triggers y tablas que se utilizaran en el sistema para asignar correctamente los permisos a los roles de administrados, analista y auditos.	Se retrasará la configuración y comprobación de los permisos, impidiendo verificar que cada usuario tenga únicamente los privilegios que necesita

2.3 Recursos por bloque

Bloque	Recursos Humanos	Recursos tecnológicos	Recursos de información
Automatización y gestión de datos	SQL Server,	Archivos Excel en formato .csv y SQL Server	Dataset de atención ambulatoria, dataset de vacunación
Seguridad y cumplimiento normativo	SQL Server,	SQL Server y GitHub para almacenar los scripst	Estructura de usuarios y permisos creados dentro de la base de datos

2.4 Contingencias ante posibles retrasos

Posible retraso	Contingencia propuesta
Problemas de calidad en los datasets: Presencia de valores nulos, duplicados y códigos especiales que dificulten la importación	Crear tablas de preparación en SQL Server que permitan conservar los valores originales.
Errores durante la importación .CSV: Incompatibilidad de formatos, tipos de datos incorrectos.	Revisar los tipos de datos y corregir el mapeo de columnas antes de importar el archivo completo.
Retrasos en el desarrollo de procedimientos, triggers o funciones: Errores en los scripts SQL	Dividir el desarrollo de los objetos entre los integrantes del grupo, realizando pruebas independientes y priorizar los componentes necesarios para completar la automatización

2.4 Procedimiento Almacenado

Antes de realizar los procedimientos se crearon 3 nuevas tablas con el propósito de no modificar los datos originales de las tablas importadas al SQL desde los archivos csv.

Tablas	Función
VacunacionProcesada	En esta tabla tiene como funcionamiento el guardar los datos de vacunación después de ser validadas
AtencionesProcesadas	En esta tabla tiene como funcionamiento el guardar los datos de las atenciones ambulatorias conservando los valores de “NE_0001” y “NE_0002”
Auditoría	En esta tabla tiene como funcionamiento el guardar información relacionada con las operaciones realizadas en las tablas procesadas

```
CREATE TABLE VacunacionProcesada (
    id INT IDENTITY(1,1) PRIMARY KEY,
    diresa VARCHAR(150),
    vacuna VARCHAR(150),
    anio INT,
    cantidad BIGINT,
    poblacion BIGINT
);


CREATE TABLE AtencionesProcesadas (
    id INT IDENTITY(1,1) PRIMARY KEY,
    ANHO INT,
    MES INT,
    UBIGEO VARCHAR(20),
    DEPARTAMENTO VARCHAR(150),
```

```
    PROVINCIA VARCHAR(150),
    DISTRITO VARCHAR(150),
    SECTOR VARCHAR(150),
    CATEGORIA VARCHAR(100),
    CO_IPRESS VARCHAR(30),
    RAZON_SOC VARCHAR(250),
    TIPO_SEXO VARCHAR(30),
    GRUPO_EDAD VARCHAR(30),
    ATENC_MEDICAS VARCHAR(30),
    ATENC_NO_MEDICAS VARCHAR(30),
    PERSONAS_ATENDIDAS VARCHAR(30)
);

CREATE TABLE LogAuditoria (
    id INT IDENTITY(1,1) PRIMARY KEY,
    tabla VARCHAR(50),
    operacion VARCHAR(30),
    descripcion VARCHAR(200),
    fecha DATETIME DEFAULT GETDATE()
);
```

Para luego implementar dos procedimientos principales para ambos archivos:

Procedimiento	Dataset	Objetivo
usp_ProcesarVacunacion	Trama_esquema_regular_descarga	Validar año, cantidades y poblacion, evitando duplicados y cargar registros procesados

usp_ProcesarAtenciones	ConsultaB1	Validar meses, identificadores, datos numéricos y códigos NE, y cargar registros procesados

El procedimiento usp_ProcesarVacunacion tiene como función automatizar la validación y el procesamiento de la información de vacunación que ya se ha importado en el SQL el archivo .csv.

```
CREATE OR ALTER PROCEDURE usp_ProcesarVacunacion
AS
BEGIN
  SET NOCOUNT ON;
  BEGIN TRY
    BEGIN TRANSACTION;
    IF EXISTS (
      SELECT 1
      FROM [Trama_esquema_regular_descarga]
      WHERE anio IS NULL
        OR cantidad IS NULL
        OR poblacion IS NULL
        OR cantidad < 0
        OR poblacion < 0
    )
      THROW 50002, 'Datos de vacunacion invalidos', 1;

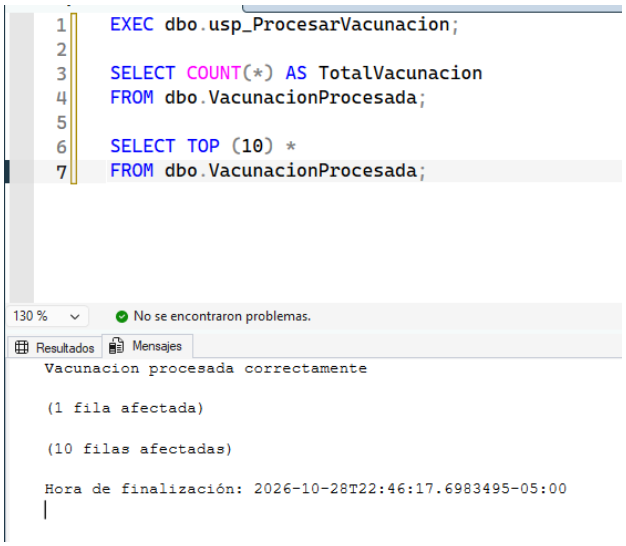
    DELETE FROM VacunacionProcesada;

    INSERT INTO VacunacionProcesada (diresa, vacuna, anio, cantidad, poblacion)
    SELECT diresa, vacuna, anio, cantidad, poblacion
    FROM [Trama_esquema_regular_descarga];
```

```
        COMMIT TRANSACTION;
        PRINT 'Vacunacion procesada correctamente';
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        PRINT ERROR_MESSAGE();
        THROW;
    END CATCH
END;
GO
```

El funcionamiento de dicho procedimiento es el siguiente:

1. Se inicia la transacción con BEGIN TRANSACTION
2. Comprueba que el año, cantidad y poblacion puede convertirse a números sin que las cantidades sean negativas
3. Elimina los registros anteriores de VacunacionProcesada
4. Inserta de nuevo los datos desde la tabla original
5. Si todo sale bien se ejecuta el comando de COMMIT siendo que se guarda los cambios
6. Si ocurre algún error se ejecuta el comando ROLLBACK, recuperando el estado inicial.



```sql
EXEC dbo.usp_ProcesarVacunacion;

SELECT COUNT(*) AS TotalVacunacion
FROM dbo.VacunacionProcesada;

SELECT TOP (10) *
FROM dbo.VacunacionProcesada;
```

130 % No se encontraron problemas.

Resultados Mensajes

	TotalVacunacion
1	5597

	id	diresa	vacuna	anio	cantidad	poblacion
1	11195	AMAZONAS	HvA	2023	3373	9923
2	11196	HUANUCO	AMA	2021	10366	14808
3	11197	HUANUCO	DPT 2° REFUERZO	2022	10306	14800
4	11198	LIMA DIRIS CENTRO	PENTA 1° DOSIS	2024	24895	24152
5	11199	SAN MARTIN	PENTA 2° DOSIS	2025	12726	13407
6	11200	TUMBES	Tdap	2024	1848	2254
7	11201	AMAZONAS	POLIO 1° DOSIS	2025	7562	8254
8	11202	CAJAMARCA	PENTA 3° DOSIS	2026	13366	20203

El segundo procedimiento denominada usp_AtencionesProcesadas procesa los datos de la tabla ConsultaB1 y los almacena en una nueva tabla denominada AtencionesProcesadas.

```sql
CREATE OR ALTER PROCEDURE usp_ProcesarAtenciones
AS
BEGIN
  SET NOCOUNT ON;

  BEGIN TRY
    BEGIN TRANSACTION;
    IF EXISTS (
      SELECT 1 FROM ConsultaB1
      WHERE MES
        NOT BETWEEN 1 AND 12
       OR MES IS NULL
       OR ANHO IS NULL
    )
```

```
        THROW 50003, 'Periodo invalido', 1;
    DELETE FROM AtencionesProcesadas;

    INSERT INTO AtencionesProcesadas
    (
        ANHO, MES, UBIGEO, DEPARTAMENTO,
        PROVINCIA, DISTRITO, SECTOR, CATEGORIA,
        CO_IPRESS, RAZON_SOC, TIPO_SEXO,
        GRUPO_EDAD, ATENC_MEDICAS,
        ATENC_NO_MEDICAS, PERSONAS_ATENDIDAS
    )
    SELECT
        ANHO,
        MES,
        UBIGEO, DEPARTAMENTO,
        PROVINCIA, DISTRITO, SECTOR, CATEGORIA,
        CO_IPRESS, RAZON_SOC, TIPO_SEXO,
        GRUPO_EDAD, ATENC_MEDICAS,
        ATENC_NO_MEDICAS, PERSONAS_ATENDIDAS
    FROM ConsultaB1;

    COMMIT TRANSACTION;

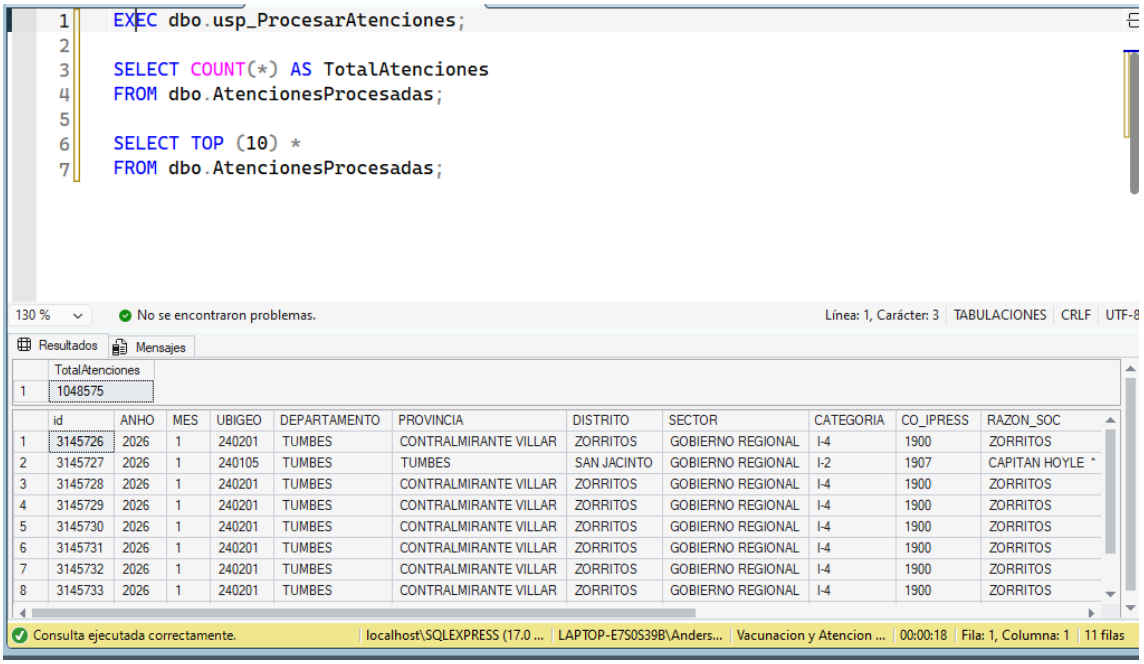
    PRINT 'Atenciones procesadas correctamente';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    PRINT ERROR_MESSAGE();
```

```
        THROW;
    END CATCH
END;
GO
```

El funcionamiento de dicho procedimiento es el siguiente:

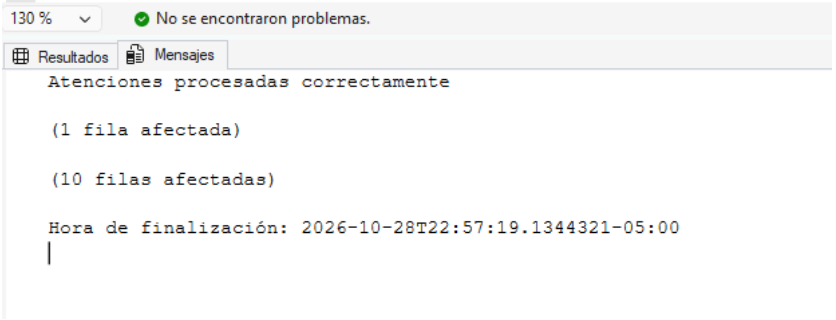
1. Inicia una transacción
2. Valida que el año sea numérico y que el mes esté entre el 1 y 12
3. Elimina los registros pasados de la tabla procesada
4. Inserta los registros originales, conservando sus columnas y códigos
5. El trigger trg.ValidarAtenciones comprueba que las cantidades sean válidas
6. Si todo sale bien se ejecuta el comando de COMMIT siendo que se guarda los cambios
7. Si ocurre algún error se ejecuta el comando ROLLBACK, recuperando el estado inicial.



```sql
EXEC dbo.usp_ProcesarAtenciones;

SELECT COUNT(*) AS TotalAtenciones
FROM dbo.AtencionesProcesadas;

SELECT TOP (10) *
FROM dbo.AtencionesProcesadas;
```



2.5 Trigger de auditoria de vacunación

Se realiza un trigger denominados “trg_AuditoriaVacunacion” para registrar automáticamente las operaciones realizadas en VacunacionProcesada.

```sql
CREATE OR ALTER TRIGGER trg_AuditoriaVacunacion
ON VacunacionProcesada
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    DECLARE @Operacion VARCHAR(30);
    IF EXISTS (SELECT 1 FROM inserted)
       AND EXISTS (SELECT 1 FROM deleted)
        SET @Operacion = 'UPDATE';
    ELSE IF EXISTS (SELECT 1 FROM inserted)
        SET @Operacion = 'INSERT';
    ELSE
        SET @Operacion = 'DELETE';

    INSERT INTO LogAuditoria
```

```
        (tabla, operacion, descripcion)
    VALUES
        ('Vacunacion Procesada',  @Operacion, 'Cambio realizado en vacunacion');
END;
GO
```

Siendo que la función del trigger es la siguiente:

1. Detecta si la operación es INSERT, UPDATE O DELETE
2. Identifica si es que el tipo de operación se ha realizado
3. Ingresa ese registro o dato a la tabla de LogAuditoria

```
1  UPDATE dbo.VacunacionProcesada
2  SET cantidad = 120
3  WHERE id = 11195;
4
5  SELECT * FROM dbo.VacunacionProcesada
6  SELECT * FROM LogAuditoria
```

130 % No se encontraron problemas. Línea:

Resultados Mensajes

	id	diresa	vacuna	anio	cantidad	poblacion
1	11195	AMAZONAS	HvA	2023	120	9923
2	11196	HUANUCO	AMA	2021	10366	14808
3	11197	HUANUCO	DPT 2° REFUERZO	2022	10306	14800
4	11198	LIMA DIRIS CENTRO	PENTA 1° DOSIS	2024	24895	24152

	id	tabla	operacion	descripcion	fecha
1	1	VacunacionProcesada	UPDATE	Cambio realizado en vacunacion	2026-10-29 14:54:27.753

2.6 Trigger de integridad de atenciones

Se realiza un trigger denominado “trg_ValidarAtenciones” que comprueba que los registros de la atención ambulatoria contengan valores permitidos, donde solamente se valida en las siguientes columnas del dataset:

- ATENC_MEDICAS
- ATENC_NO_MEDICAS
- PERSONAS_ATENDIDAS

```
CREATE OR ALTER TRIGGER trg_ValidarAtenciones
ON AtencionesProcesadas
AFTER INSERT, UPDATE
AS
BEGIN
  IF EXISTS (SELECT 1 FROM inserted
    WHERE
      ATENC_MEDICAS IS NULL
      OR ATENC_NO_MEDICAS IS NULL
      OR PERSONAS_ATENDIDAS IS NULL

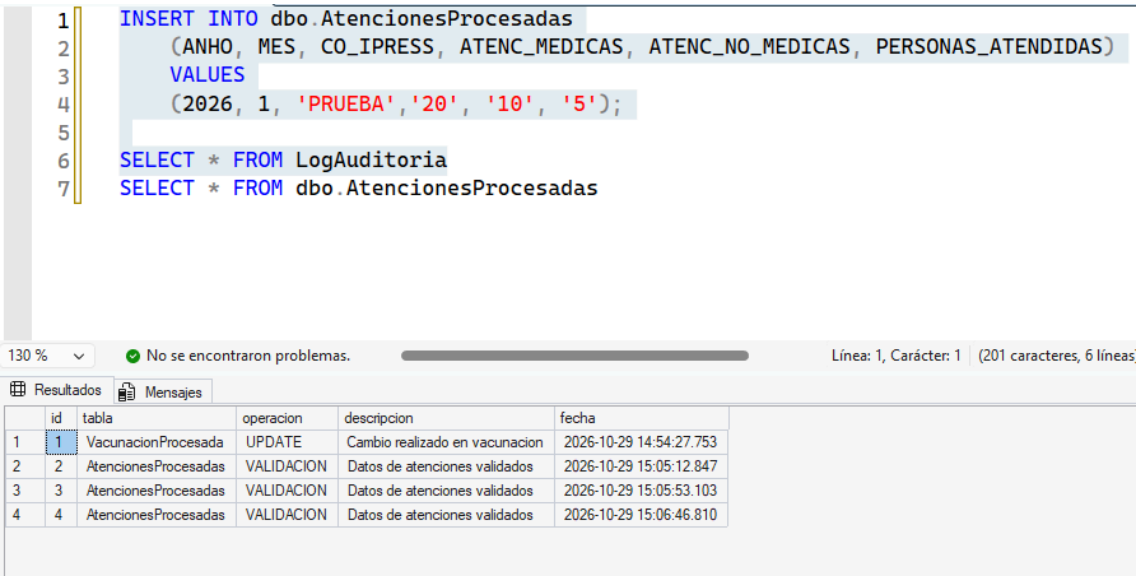
      OR (ATENC_MEDICAS NOT IN ('NE_0001','NE_0002')
       AND (TRY_CONVERT(BIGINT, ATENC_MEDICAS) IS NULL
        OR TRY_CONVERT(BIGINT, ATENC_MEDICAS) < 0))
      OR (ATENC_NO_MEDICAS NOT IN ('NE_0001','NE_0002')
       AND (TRY_CONVERT(BIGINT, ATENC_NO_MEDICAS) IS NULL
        OR TRY_CONVERT(BIGINT, ATENC_NO_MEDICAS) < 0))
      OR (PERSONAS_ATENDIDAS NOT IN ('NE_0001','NE_0002')
       AND (TRY_CONVERT(BIGINT, PERSONAS_ATENDIDAS) IS NULL
        OR TRY_CONVERT(BIGINT, PERSONAS_ATENDIDAS) < 0))
  )
  BEGIN
    ROLLBACK TRANSACTION;
    THROW 50001, 'Atenciones invalidas', 1;
  END;

  INSERT INTO LogAuditoria
```

```
    (tabla, operacion, descripcion)
  VALUES ('Atenciones Procesadas', 'VALIDACION', 'Datos de atenciones validados');
END;
GO
```

El trigger funciona de la siguiente manera:

1. Verifica los datos nuevos ingresados o modificados
2. Si encuentra algún valor erróneo se ejecuta el comando ROLLBACK TRANSACTION y genera un error personalizado con THROW
3. Si encuentra que todos los valores son válidos se registra la operación válida en LogAuditoria



```
INSERT INTO dbo.AtencionesProcesadas
    (ANHO, MES, CO_IPRESS, ATENC_MEDICAS, ATENC_NO_MEDICAS, PERSONAS_ATENDIDAS)
    VALUES
    (2026, 1, 'PRUEBA','20', '10', '5');

SELECT * FROM LogAuditoria
SELECT * FROM dbo.AtencionesProcesadas
```

130 % — No se encontraron problemas. — Línea: 1, Carácter: 1 — (201 caracteres, 6 líneas)

Resultados | Mensajes

	id	tabla	operacion	descripcion	fecha
1	1	VacunacionProcesada	UPDATE	Cambio realizado en vacunacion	2026-10-29 14:54:27.753
2	2	AtencionesProcesadas	VALIDACION	Datos de atenciones validados	2026-10-29 15:05:12.847
3	3	AtencionesProcesadas	VALIDACION	Datos de atenciones validados	2026-10-29 15:05:53.103
4	4	AtencionesProcesadas	VALIDACION	Datos de atenciones validados	2026-10-29 15:06:46.810

2.7 Función Escalar

Se ha creado una función denominada “fn_CodigoNE”, permitiendo interpretar el significado de los códigos específicos que aparecen en algunas columnas de los datos de atención ambulatoria.

```
 CREATE OR ALTER FUNCTION fn_CodigoNE (@Codigo VARCHAR(30))
RETURNS VARCHAR(100)
AS
BEGIN
  RETURN CASE
    WHEN @Codigo = 'NE_0001'
```

```
            THEN 'Actividad no realizada en la IPRESS'
        WHEN @Codigo = 'NE_0002'
            THEN 'Actividad no realizada en el periodo'
        ELSE 'Sin codigo especial'
    END;
END;
GO
```

```
SELECT
    dbo.fn_CodigoNE('NE_0001') AS Codigo_1,
    dbo.fn_CodigoNE('NE_0002') AS Codigo_2,
    dbo.fn_CodigoNE('150') AS Valor_numerico;
```

130 % No se encontraron problemas.

Resultados Mensajes

	Codigo_1	Codigo_2	Valor_numerico
1	Actividad no realizada en la IPRESS	Actividad no realizada en el periodo	Sin codigo especial

2.8 Justificación de diseño

Para la realización del proyecto se implementó la lógica en SQL Server mediante procedimientos almacenados, triggers y funciones sin depender de la lógica en una aplicación. Debido a que el sistema necesita garantizar la integridad, consistencia y seguridad de los datos durante su procedimiento. Los procedimientos implementados tienen como función procesar y validar información de vacunación, los triggers son los que permiten registrar de manera automática operaciones esenciales como los de INSERT, UPDATE y DELETE, como a su vez validar los datos de atenciones y por último las funciones permiten interpretar los códigos especiales que aparecen en las atenciones ambulatorias como lo son NE_001 y NE_002.

A diferencia de implementar las reglas de una aplicación, centralizar la lógica en la base de datos nos permite que las validaciones y auditorías se implementen independientemente del programa que acceda a la información. Además, que facilita la reutilización de las reglas y el control de errores mediante las transacciones y la utilización del TRY…CATCH. Por ello, se considera que este diseño es adecuado para el proyecto debido a que mantiene que la información esta ordenada y controlada.

3. Seguridad y cumplimiento normativo conforme a la Ley N.° 29733

3.1 Fundamentación y comparación de alternativas para la automatización

Alternativa	Fortaleza posible	Limitación posible	Cuando debería ser posible
Lógica principal en procedimientos almacenados y triggers SQL Server	Permite validar los datos directamente en la base de datos, controlar operaciones mediante transacciones y registrar los cambios realizados por el usuario mediante la tabla de auditoria	Triggers complejos que pueden ser difíciles para el mantenimiento de la base de datos	Cuando se necesite controlar la integridad de los datos, registrar las modificaciones y asegurar que las reglas se apliquen directamente a la base de datos
Lógica en aplicación	Permite revisar los archivos antes de importarlos, mostrar mensajes de error y ofrecer una interfaz para que los usuarios gestionen la información	La regla de validación puede quedar fuera de la base de datos. Si existen diferentes aplicaciones que modifican la información, cada una tendría que aplicar las mismas reglas	Cuando se necesita una interfaz de usuario, validaciones previas a la importación o procesos que dependan de las acciones realizadas por los usuarios

3.2 Justificación de la alternativa

Se eligió la alternativa basada en procedimientos almacenados, triggers y funciones en SQL Server, porque el proyecto necesita controlar la carga y validación de dos conjuntos de datos correspondiente a la atención de la salud y el otro con la vacunación.

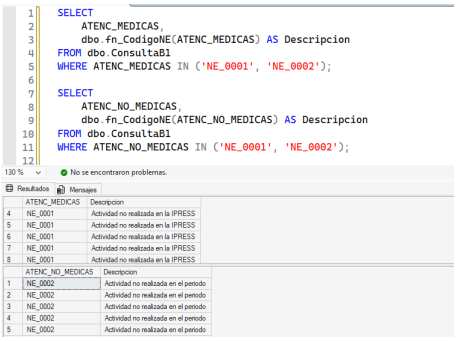
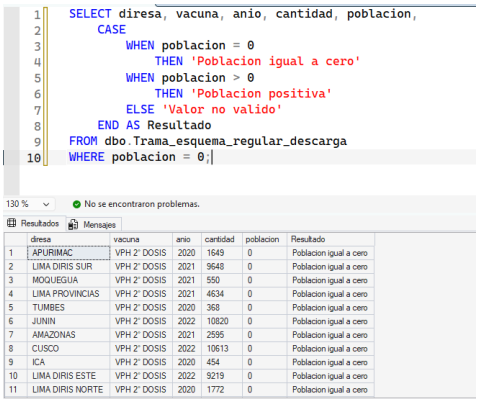
Siendo que al utilizar el SQL Server como alternativa podemos comprobar los registros antes de incorporarlos a las tablas originales. Siendo que en la dataset de atención de la salud se deben identificar valores especiales como el NE_0001 y NE_0002, evitando que se conviertan incorrectamente en valores numéricos y en la segunda datase sobre la vacunación se debe controlar campos como la cantidad de vacunas y población evitando realizar cálculos cuando la población sea igual a 0.

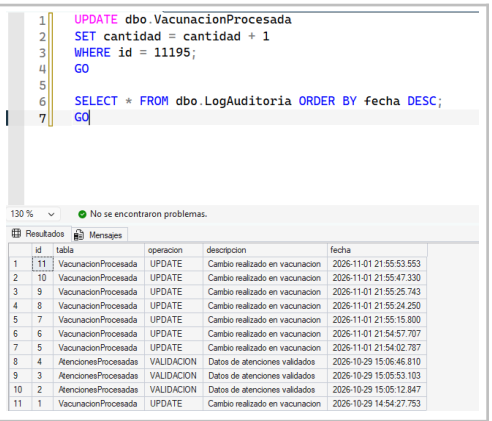
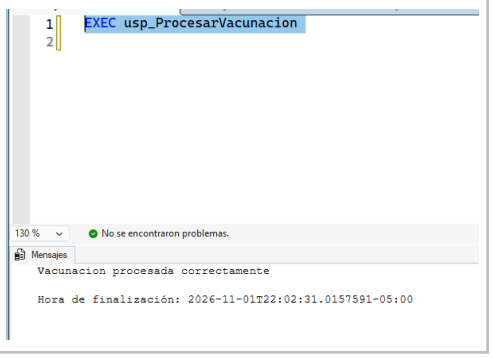
A su vez que los procedimientos almacenados permitirán organizar las operaciones de carga y validación, los triggers ayudarán a registrar modificaciones, mientras que las funciones permitirán las validaciones dentro de las consultas.

Por lo tanto, el uso de SQL Server en este proyecto responde a la necesidad de controlar las operaciones y mantener la trazabilidad de los datos.

3.3 Criterios que deben medirse

Criterio técnico	Cómo medirlo en la prueba	Resultado
Integridad de los datos	Se realizarán pruebas con registros válidos e inválidos para verificar que los procedimientos acepten los datos correctos e identifiquen los que no cumplen las reglas establecidas. También se verificará los códigos NE_0001 y NE_0002 al igual que los registros de poblacion = 0	SELECT ATENC_MEDICAS, dbo.fn_CodigoNE(ATENC_MEDICAS) AS Descripcion FROM dbo.ConsultaB1 WHERE ATENC_MEDICAS IN ('NE_0001', 'NE_0002'); SELECT ATENC_NO_MEDICAS, dbo.fn_CodigoNE(ATENC_NO_MEDICAS) AS Descripcion FROM dbo.ConsultaB1 WHERE ATENC_NO_MEDICAS IN ('NE_0001', 'NE_0002'); SELECT

		PERSONAS_ATENDIDAS, dbo.fn_CodigoNE(PERSONAS_ATENDIDAS) AS Descripcion FROM dbo.ConsultaB1 WHERE PERSONAS_ATENDIDAS IN ('NE_0001', 'NE_0002');  SELECT diresa, vacuna, anio, cantidad, poblacion, CASE WHEN poblacion = 0 THEN 'Poblacion igual a cero' WHEN poblacion > 0 THEN 'Poblacion positiva' ELSE 'Valor no valido' END AS Resultado FROM dbo.Trama_esquema_regular_descarga WHERE poblacion = 0; 
Trazabilidad de las operaciones	Se ejecutaron operaciones como actualización para verificar que los triggers funcionen correctamente y registre dicha acción en la tabla de auditoría, permitiendo identificar	UPDATE dbo.VacunacionProcesada SET cantidad = cantidad + 1 WHERE id = 11195; GO SELECT * FROM dbo.LogAuditoria ORDER BY fecha DESC; GO

	qué operación se realizó y cuando ocurrió	
Recuperación ante un error	Provocar un error controlado y comprobar el que el procedimiento almacenado gestione la excepción mediante TRY…CATCH y ejecuta el ROLLBACK cuando corresponda	EXEC usp_ProcesarVacunacion 

3.4 Creación de tres roles de base de datos

Rol	Permisos	Justificación
Administrador	Consultar, insertar, actualizar y eliminar registros, a su vez ejecutar procedimientos de procesamiento	Se encarga del mantenimiento y procesamiento de la base de datos
Analista de datos	Solo consultar las tablas procesadas	Puede elaborar reportes sin modificar ni eliminar información por error
Auditor	Solo consultar los registros de auditoria	Puede revisar las operaciones realizadas y verificar la seguridad

```
CREATE ROLE Administrador;

CREATE ROLE Analista;

CREATE ROLE Auditor;

GO
```

```
GRANT SELECT, INSERT, UPDATE, DELETE
ON dbo.VacunacionProcesada
TO Administrador;

GRANT SELECT, INSERT, UPDATE, DELETE
ON dbo.AtencionesProcesadas
TO Administrador;

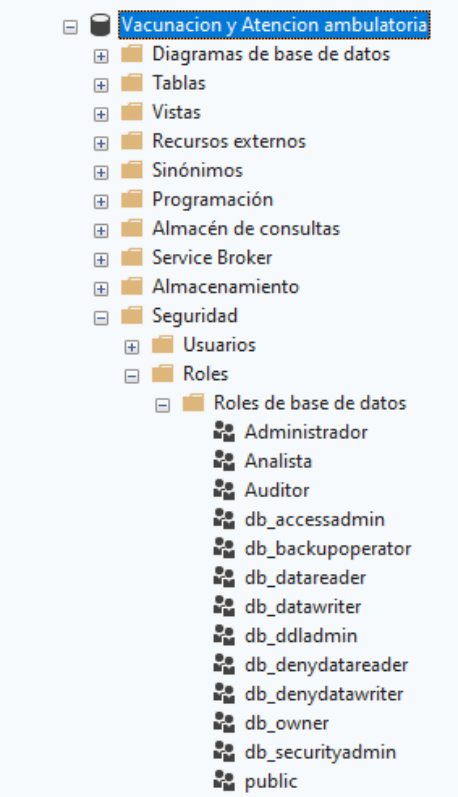
GRANT EXECUTE ON dbo.usp_ProcesarVacunacion
TO Administrador;

GRANT EXECUTE ON dbo.usp_ProcesarAtenciones
TO Administrador;
GO

GRANT SELECT ON dbo.VacunacionProcesada
TO Analista;

GRANT SELECT ON dbo.AtencionesProcesadas
TO Analista;
GO

GRANT SELECT ON dbo.LogAuditoria
TO Auditor;
GO
```




```
22  GRANT SELECT ON dbo.VacunacionProcesada
23  TO Analista;
24
25  GRANT SELECT ON dbo.AtencionesProcesadas
26  TO Analista;
27  GO
28
29
30  GRANT SELECT ON dbo.LogAuditoria
31  TO Auditor;
32  GO
```

130 % No se encontraron problemas.

Mensajes

```
Los comandos se han completado correctamente.

Hora de finalización: 2026-11-01T22:05:33.9631981-05:00
```

3.5 Política de respaldo y restauración

Elemento	Política propuesta
Tipo de respaldo	Completo y diferencial
Respaldo completo	Una vez por semana
Respaldo diferencial	3 veces por semana
Ubicación	Carpeta segura destinada a respaldos

Protección	Acceso restringido y copias protegidas

```
BACKUP DATABASE [Vacunacion y Atencion ambulatoria]

TO DISK = 'C:\Vacunacion_y_Atencion\Vacunacion y Atencion ambulatoria Completa.bak'

WITH INIT, STATS = 10, NAME = 'Backup completo de Vacunacion y atencion';

GO


BACKUP DATABASE [Vacunacion y Atencion ambulatoria]

TO DISK = 'C:\Vacunacion_y_Atencion\Vacunacion_y_Atencion_diferencial\Vacunacion y Atencion ambulatoria Diferencial.bak'

WITH DIFFERENTIAL, INIT, STATS = 10,

        NAME = 'Backup diferencial de Vacunacion y atencion';

GO

RESTORE HEADERONLY

FROM DISK = 'C:\Vacunacion_y_Atencion\Vacunacion y Atencion ambulatoria Completa.bak';

GO


RESTORE HEADERONLY

FROM DISK = 'C:\Vacunacion_y_Atencion\Vacunacion_y_Atencion_diferencial\Vacunacion y Atencion ambulatoria Diferencial.bak';

GO
```

```sql
BACKUP DATABASE [Vacunacion y Atencion ambulatoria]
TO DISK = 'C:\Vacunacion_y_Atencion\Vacunacion y Atencion ambulatoria Completa.bak'
WITH INIT, STATS = 10, NAME = 'Backup completo de Vacunacion y atencion';
GO

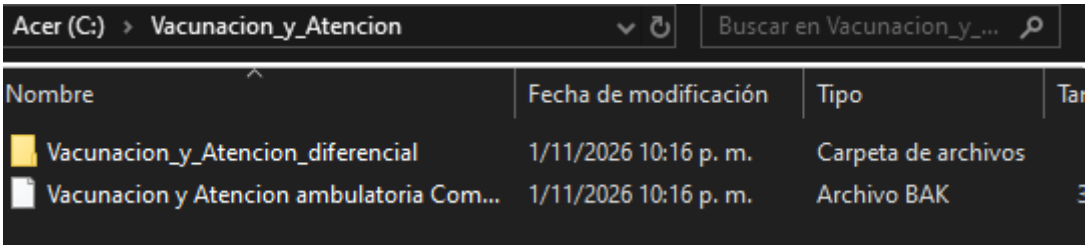
BACKUP DATABASE [Vacunacion y Atencion ambulatoria]
TO DISK = 'C:\Vacunacion_y_Atencion\Vacunacion_y_Atencion_diferencial\Vacunacion y Atencion
```

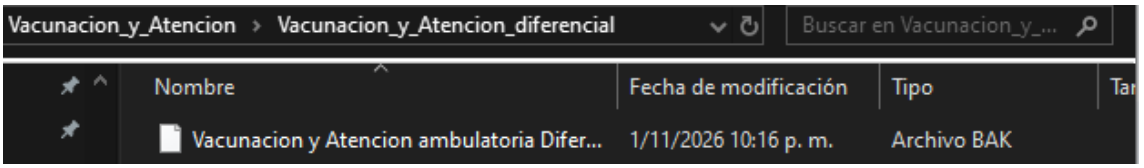
130 % — No se encontraron problemas. — Línea: 1, Carácter: 1 | (493 caracteres, 10 líneas) | TABULACIONES | CRLF | UTF-8

Mensajes

```
40 percent processed.
50 percent processed.
60 percent processed.
70 percent processed.
80 percent processed.
90 percent processed.
100 percent processed.
Processed 39224 pages for database 'Vacunacion y Atencion ambulatoria', file 'Vacunacion y Atencion ambulatoria'
Processed 1 pages for database 'Vacunacion y Atencion ambulatoria', file 'Vacunacion y Atencion ambulatoria_log'
BACKUP DATABASE successfully processed 39225 pages in 0.868 seconds (353.040 MB/sec).
16 percent processed.
24 percent processed.
32 percent processed.
41 percent processed.
57 percent processed.
65 percent processed.
74 percent processed.
```

130 % — No se encontraron problemas. — Línea: 29, Carácter: 1 | TABULACIONES | MIXTO | UTF-8 with BOM






```sql
GO


RESTORE HEADERONLY
FROM DISK = 'C:\Vacunacion_y_Atencion\Vacunacion y Atencion ambulatoria Completa.bak';
GO

RESTORE HEADERONLY
FROM DISK = 'C:\Vacunacion_y_Atencion\Vacunacion_y_Atencion_diferencial\Vacunacion y Atenci
GO
```

130 % — No se encontraron problemas. — Línea: 13, Carácter: 1 | (261 caracteres, 7 líneas) | TABULACIONES | CRLF | UTF-8

Resultados | Mensajes

	BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType	UserName	ServerName
1	Backup completo de Vacunacion y atencion	NULL	1	NULL	0	1	2	LAPTOP-E7S0S39B\Anderson	LAPTOP-E7S0S39B\S

	BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType	UserName	ServerName
1	Backup diferencial de Vacunacion y atencion	NULL	5	NULL	0	1	2	LAPTOP-E7S0S39B\Anderson	LAPTOP-E7S0S39B\S

3.6 Análisis del rendimiento antes y después de crear un índice

El índice dentro de la base de datos es una estructura de datos que acelera la búsqueda de información de la tabla principal. En este proyecto se utilizarán los índices y también se mostrará los cambios que existen antes y después de ejecutar el comando.

Consulta antes del índice:

```
SET STATISTICS IO ON;

SET STATISTICS TIME ON;

GO


SELECT

   DEPARTAMENTO,

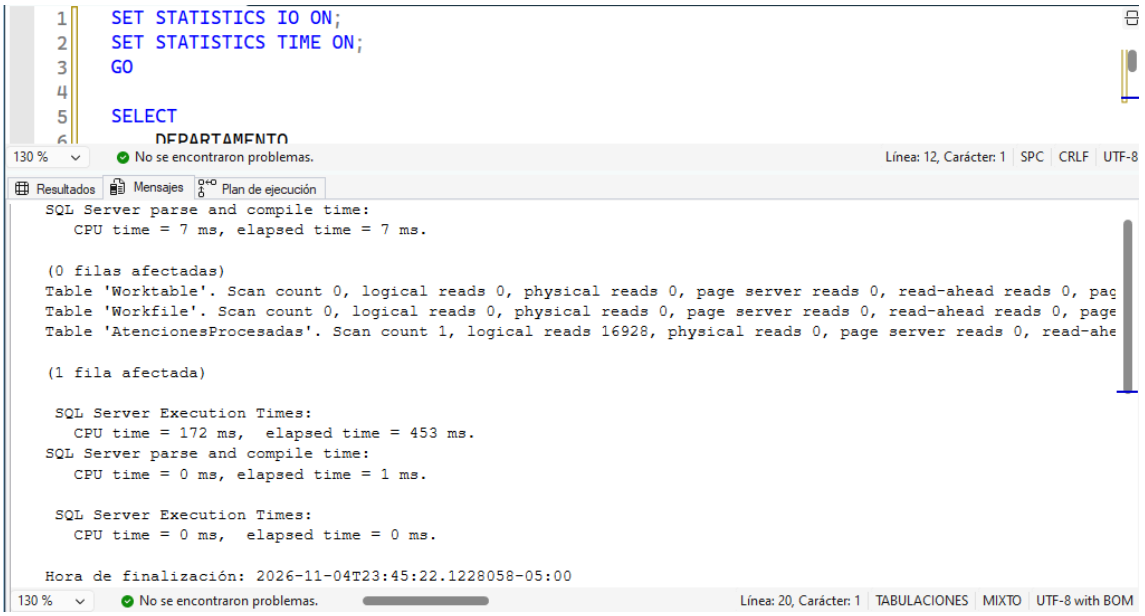
   COUNT(*) AS TotalRegistros

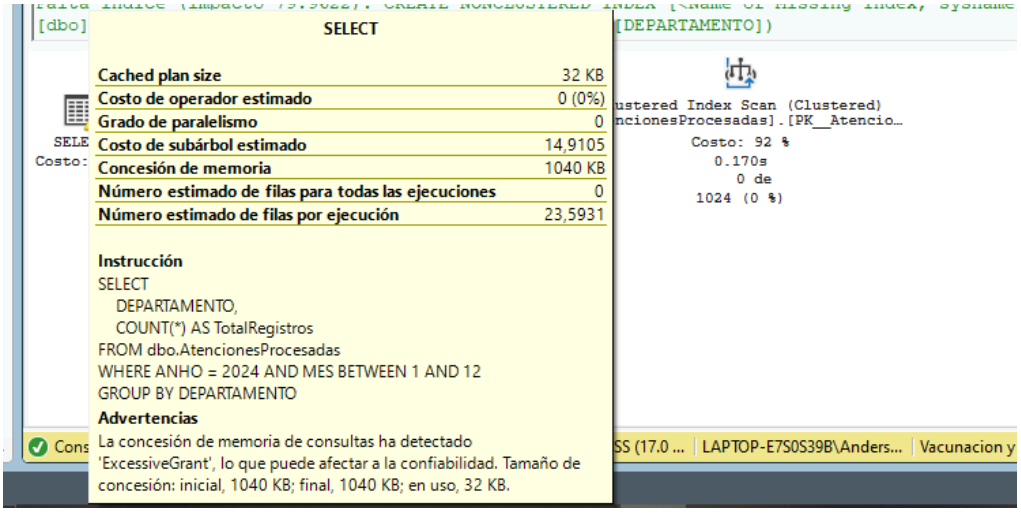
FROM dbo.AtencionesProcesadas

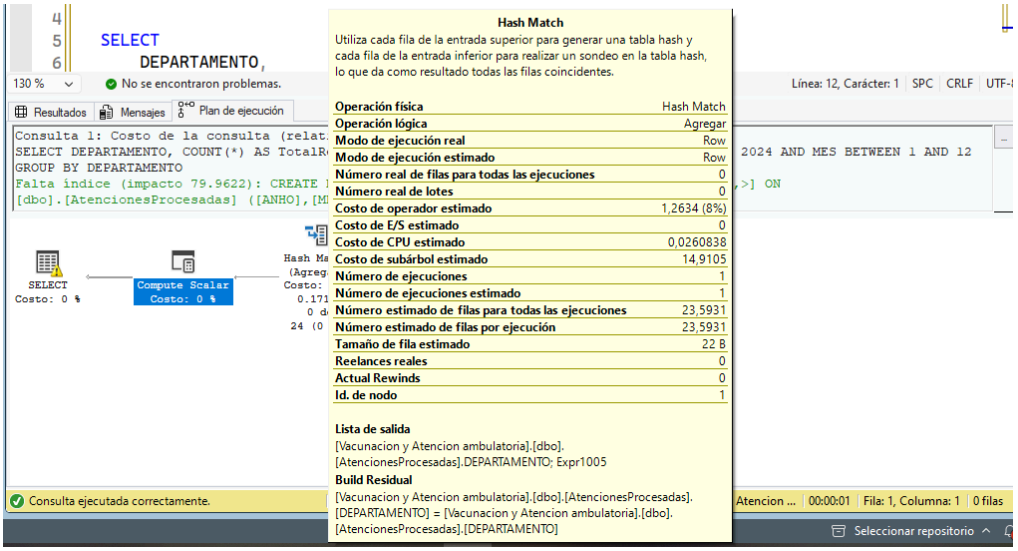
WHERE ANHO = 2024 AND MES BETWEEN 1 AND 12

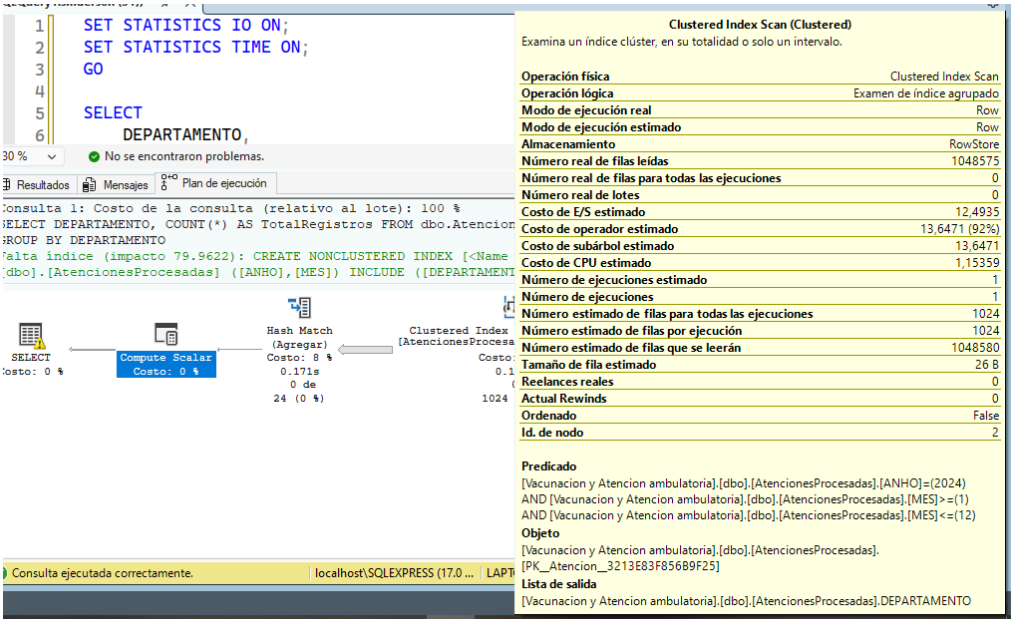
GROUP BY DEPARTAMENTO;

GO
```



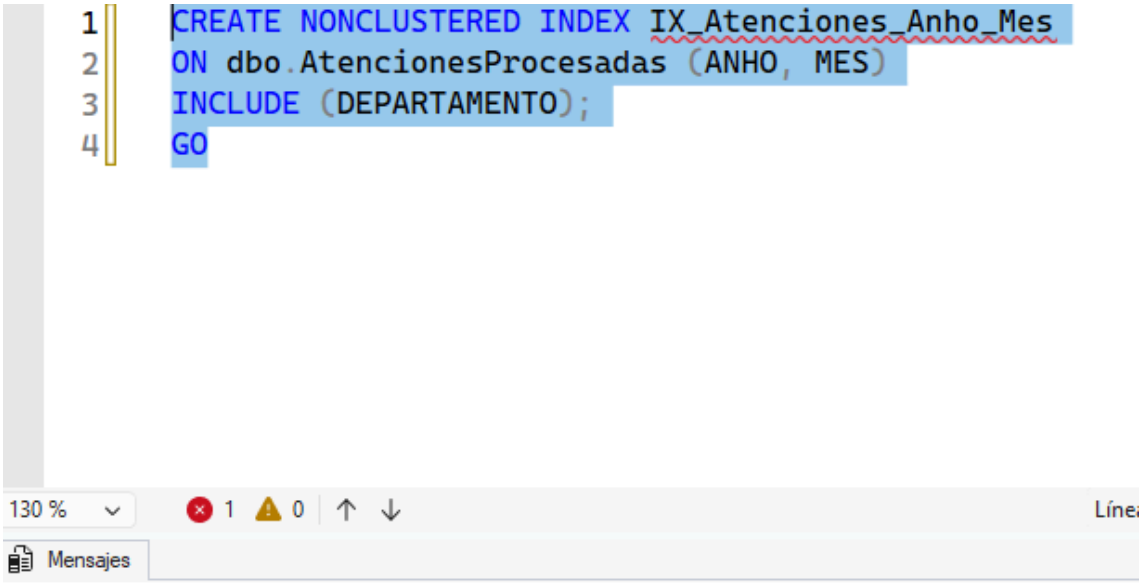






Crear un índice:

```
CREATE NONCLUSTERED INDEX IX_Atenciones_Anho_Mes
ON dbo.AtencionesProcesadas (ANHO, MES)
INCLUDE (DEPARTAMENTO);
GO
```



Ejecutar otra vez la misma consulta:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT
   DEPARTAMENTO,
   COUNT(*) AS TotalRegistros
FROM dbo.AtencionesProcesadas
WHERE ANHO = 2024
 AND MES BETWEEN 1 AND 12
GROUP BY DEPARTAMENTO;
GO


SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

GO

```sql
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
SELECT
    DEPARTAMENTO,
    COUNT(*) AS TotalRegistros
FROM dbo.AtencionesProcesadas
WHERE ANHO = 2024
  AND MES BETWEEN 1 AND 12
```

130 % — No se encontraron problemas. — Línea: 2, Carácter: 24 | SPC | CRLF | UTF-8

Resultados | Mensajes | Plan de ejecución

```
(0 filas afectadas)
Table 'AtencionesProcesadas'. Scan count 1, logical reads 3, physical reads 0, page server reads 0, read-ahead r

(1 fila afectada)

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 30 ms.
SQL Server parse and compile time:
   CPU time = 0 ms, elapsed time = 0 ms.

 SQL Server Execution Times:
   CPU time = 0 ms,  elapsed time = 0 ms.

Hora de finalización: 2026-11-05T00:00:14.6106504-05:00
```

130 % — No se encontraron problemas. — Línea: 16, Carácter: 1 | TABULACIONES | MIXTO | UTF-8 with BOM

```sql
GO
```

130 % — No se encontraron problemas. — Línea: 12, Carác

SELECT

Cached plan size	32 KB
Costo de operador estimado	0 (0%)
Grado de paralelismo	0
Costo de subárbol estimado	0,0146455
Concesión de memoria	1024 KB
Número estimado de filas para todas las ejecuciones	0
Número estimado de filas por ejecución	1

Instrucción

```
SELECT
   DEPARTAMENTO,
   COUNT(*) AS TotalRegistros
FROM dbo.AtencionesProcesadas
WHERE ANHO = 2024
  AND MES BETWEEN 1 AND 12
GROUP BY DEPARTAMENTO
```

Sort (Ordenar) — Costo: 78 % — 0.000s — 0 de 1 (0 %)

Index Seek (NonClustered) [AtencionesProcesadas].[IX_Atencion… — Costo: 22 % — 0.000s — 0 de 1 (0 %)

LAPTOP-E7S0S39B\Anders... | Vacunacion y Atencion ... | 00:00:00

```sql
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

130 % — Línea: 12

Resultados | Mensajes

```
Consulta 1: Costo de
SELECT DEPARTAMENTO,
GROUP BY DEPARTAMENT
```

Compute Scalar

Calcula valores nuevos a partir de los valores existentes de una fila.

Operación física	Compute Scalar
Operación lógica	Compute Scalar
Modo de ejecución estimado	Row
Costo de operador estimado	0 (0%)
Costo de E/S estimado	0
Costo de subárbol estimado	0,0146455
Costo de CPU estimado	0
Número de ejecuciones estimado	1
Número estimado de filas por ejecución	1
Número estimado de filas para todas las ejecuciones	1
Tamaño de fila estimado	22 B
Id. de nodo	0

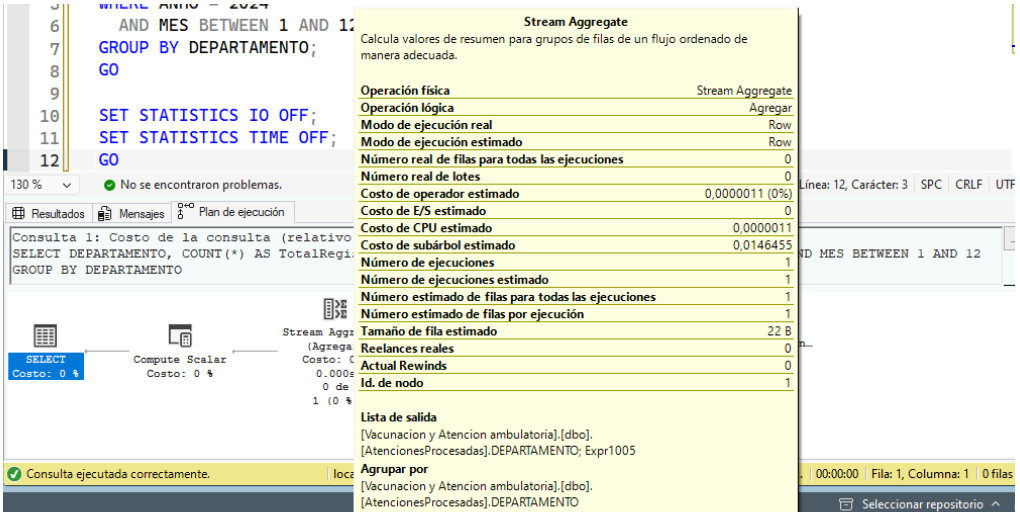
Lista de salida

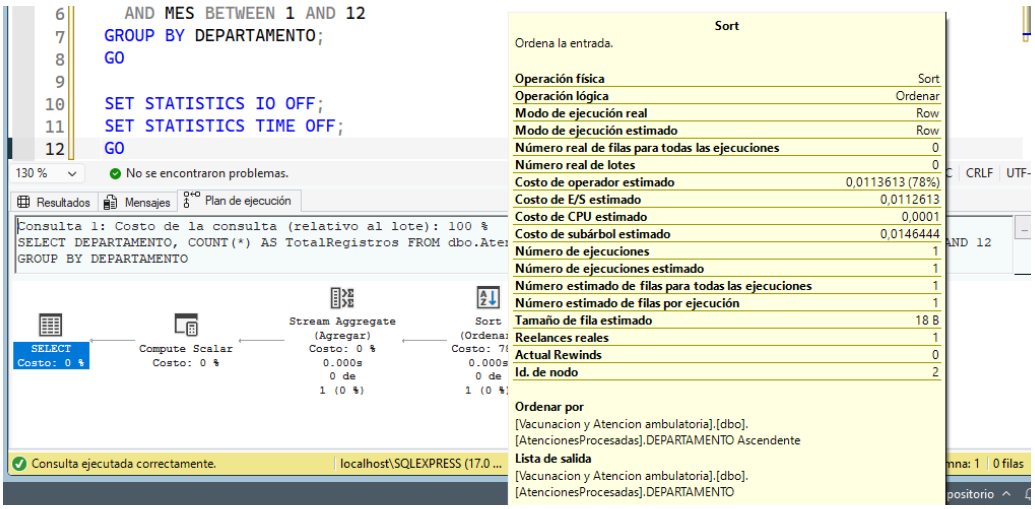
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO; Expr1002

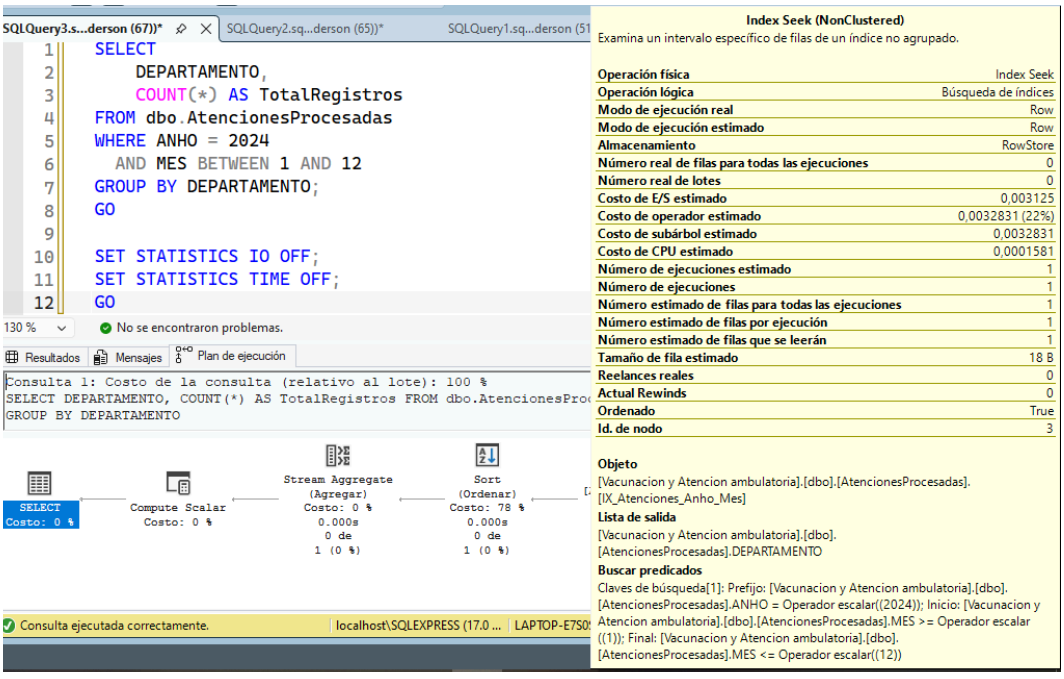
SELECT Costo: 0 %

Index Seek (NonClustered) — Costo: 22 % — 0.000s — 0 de 1 (0 %)

Consulta ejecutada correcta | Vacunacion y Atencion ... | 00:00:


AND MES BETWEEN 1 AND 12
GROUP BY DEPARTAMENTO;
GO
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
No se encontraron problemas.
Resultados
Mensajes
Plan de ejecución
Consulta 1: Costo de la consulta (relativo
SELECT DEPARTAMENTO, COUNT(*) AS TotalRegi
GROUP BY DEPARTAMENTO
SELECT
Costo: 0 %
Compute Scalar
Costo: 0 %
Consulta ejecutada correctamente.
Stream Aggregate
Calcula valores de resumen para grupos de filas de un flujo ordenado de manera adecuada.
Operación física: Stream Aggregate
Operación lógica: Agregar
Modo de ejecución real: Row
Modo de ejecución estimado: Row
Número real de filas para todas las ejecuciones: 0
Número real de lotes: 0
Costo de operador estimado: 0,0000011 (0%)
Costo de E/S estimado: 0
Costo de CPU estimado: 0,0000011
Costo de subárbol estimado: 0,0146455
Número de ejecuciones: 1
Número de ejecuciones estimado: 1
Número estimado de filas para todas las ejecuciones: 1
Número estimado de filas por ejecución: 1
Tamaño de fila estimado: 22 B
Reelances reales: 0
Actual Rewinds: 0
Id. de nodo: 1
Lista de salida
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO; Expr1005
Agrupar por
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO


AND MES BETWEEN 1 AND 12
GROUP BY DEPARTAMENTO;
GO
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
No se encontraron problemas.
Resultados
Mensajes
Plan de ejecución
Consulta 1: Costo de la consulta (relativo al lote): 100 %
SELECT DEPARTAMENTO, COUNT(*) AS TotalRegistros FROM dbo.Ate
GROUP BY DEPARTAMENTO
SELECT
Costo: 0 %
Compute Scalar
Costo: 0 %
Stream Aggregate
(Agregar)
Costo: 0 %
0.000s
0 de
1 (0 %)
Sort
(Ordena
Costo: 7
0.000s
0 de
1 (0 %)
Consulta ejecutada correctamente.
localhost\SQLEXPRESS (17.0 ...
Sort
Ordena la entrada.
Operación física: Sort
Operación lógica: Ordenar
Modo de ejecución real: Row
Modo de ejecución estimado: Row
Número real de filas para todas las ejecuciones: 0
Número real de lotes: 0
Costo de operador estimado: 0,0113613 (78%)
Costo de E/S estimado: 0,0112613
Costo de CPU estimado: 0,0001
Costo de subárbol estimado: 0,0146444
Número de ejecuciones: 1
Número de ejecuciones estimado: 1
Número estimado de filas para todas las ejecuciones: 1
Número estimado de filas por ejecución: 1
Tamaño de fila estimado: 18 B
Reelances reales: 1
Actual Rewinds: 0
Id. de nodo: 2
Ordenar por
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO Ascendente
Lista de salida
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO


SQLQuery3.s...derson (67))*
SQLQuery2.sq...derson (65))*
SQLQuery1.sq...derson (51
SELECT
DEPARTAMENTO,
COUNT(*) AS TotalRegistros
FROM dbo.AtencionesProcesadas
WHERE ANHO = 2024
AND MES BETWEEN 1 AND 12
GROUP BY DEPARTAMENTO;
GO
SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
No se encontraron problemas.
Resultados
Mensajes
Plan de ejecución
Consulta 1: Costo de la consulta (relativo al lote): 100 %
SELECT DEPARTAMENTO, COUNT(*) AS TotalRegistros FROM dbo.AtencionesProc
GROUP BY DEPARTAMENTO
SELECT
Costo: 0 %
Compute Scalar
Costo: 0 %
Stream Aggregate
(Agregar)
Costo: 0 %
0.000s
0 de
1 (0 %)
Sort
(Ordenar)
Costo: 78 %
0.000s
0 de
1 (0 %)
Consulta ejecutada correctamente.
localhost\SQLEXPRESS (17.0 ...
LAPTOP-E7S0
Index Seek (NonClustered)
Examina un intervalo específico de filas de un índice no agrupado.
Operación física: Index Seek
Operación lógica: Búsqueda de índices
Modo de ejecución real: Row
Modo de ejecución estimado: Row
Almacenamiento: RowStore
Número real de filas para todas las ejecuciones: 0
Número real de lotes: 0
Costo de E/S estimado: 0,003125
Costo de operador estimado: 0,0032831 (22%)
Costo de subárbol estimado: 0,0032831
Costo de CPU estimado: 0,0001581
Número de ejecuciones estimado: 1
Número de ejecuciones: 1
Número estimado de filas para todas las ejecuciones: 1
Número estimado de filas por ejecución: 1
Número estimado de filas que se leerán: 1
Tamaño de fila estimado: 18 B
Reelances reales: 0
Actual Rewinds: 0
Ordenado: True
Id. de nodo: 3
Objeto
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].[IX_Atenciones_Anho_Mes]
Lista de salida
[Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].DEPARTAMENTO
Buscar predicados
Claves de búsqueda[1]: Prefijo: [Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].ANHO = Operador escalar((2024)); Inicio: [Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].MES >= Operador escalar((1)); Final: [Vacunacion y Atencion ambulatoria].[dbo].[AtencionesProcesadas].MES <= Operador escalar((12))

Métrica	Antes del índice	Después del índice
Lecturas lógicas	16 928	3
Tiempo de CPU	172 ms	0 ms
Tiempo transcurrido	453 ms	30 ms

Teniendo en cuenta los resultados que obtuvimos antes y después del implementar el índice, se puede decir que hubo una mejora en las métricas obtenidas. Ya que inicialmente SQL Server registro 16928 lecturas lógicas, pero después de realizar los índices disminuyo a solo 3 lecturas. Para que luego en el tiempo del CPU que tuvo un resultado de 172ms paso a 0ms y el tiempo transcurrido se redujo 453ms a 30ms.

Los resultados que se obtuvieron después de la creación del índice tuvieron un impacto positivo al optimizar el acceso a los datos, reduciendo la cantidad de lecturas y también el tiempo de ejecución de la consulta. Por ende, la utilización de índices en las bases de datos mejora el rendimiento de las operaciones de búsqueda siempre y cuando se tenga un buen uso de este recurso ya que sino puede haber problemas dentro del sistema como el consumo excesivo de recursos.

4 Integración SQL–NoSQL: análisis comparativo y decisión arquitectónica

4.1 Operaciones CRUD en MongoDB

Las operaciones CRUD nos permiten demostrar las acciones de crear, leer, actualizar y eliminar documentos

Operación	Comando MongoDB	Finalidad
Create (crear)	insertOne() / insertMany()	Registrar documentes desde las fuentes de atención y vacunación
Read (leer)	find() / findOne()	Consultar atenciones y vacunaciones por DIRESA, vacuna o año
Update (actualizar)	updateOne() / updateMany()	Actualiza el estado de revisión o de una observación teniendo como ventaja que si actualizas con updateOne() y no

		especificas solo se actualiza el primer dato
Delete (eliminar)	deleteOne() / deleteMany()	Elimina una o mas registros de la prueba autorizada

4.2 Comparación SQL Server frente a MongoDB

La comparación se realiza considerando las necesidades del proyecto ya sea almacenar registros de ambas fuentes, conservar los valores especiales, validar la información y consultar ya sea por territorio o periodo, y facilitar las futuras ampliaciones del modelo. Se consideran 5 criterios técnicos. La ponderación y clasificación son una valoración inicial del equipo y se deben de revisarse después de ejecutar las mismas consultas con el mismo conjunto de prueba en ambos motores de base de datos.

Criterio técnico	SQL Server (1-5)	MongoDB (1-5)	Justificación
Modelo de datos y flexibilidad	4	5	En SQL Server organiza los datos tabulares en columnas definidas mientras que en MongoDB facilita documentos con subdocumentos y campos opcionales
Integridad y consistencia	5	4	SQL Server ofrece restricciones relacionales y transacciones mientras que MongoDB también permite validaciones y transacciones, aunque este exige definirlas y aplicarlas en el

			diseño documental
Consultas y agregaciones	5	4	SQL Server es adecuado para consultas relacionales, agrupaciones y cruces mientras que MongoDB dispone del find para consultas documentales
Escalabilidad	4	5	En MongoDB se incorpora opciones de distribución horizontal mientras que en SQL Server dispone de alternativas de escalamiento según edición y arquitectura
Latencia y rendimiento de consulta	4	4	Se debe compararse con el mismo volumen, filtros, índices, hardware y numero de ejecuciones por ende no se asigna una ventaja medida

Con la valoración preliminar SQL Server obtiene 22 puntos mientras que MongoDB tiene

4.3 Decisión arquitectónica

Se propone una arquitectura hibrida con SQL Server como repositorio principal de los datos estructurados y MongoDB como componente complementario para

explorar y almacenar documentos con atributos variables. Esta decisión permite cumplir con la exploración NoSQL solicitada sin reemplazar innecesariamente al motor relacional que soporta los procesos de validación, integridad, roles y auditoria del proyecto.

Componente	Responsabilidad propuesta	Motivo
SQL Server	Conservar las tablas estructuradas de atenciones y vacunación, aplicar reglas de integridad, procedimiento, triggers, roles y auditoria	Las fuentes son tabulares y requieren control de calidad, consistencia y trazabilidad
MongoDB	Almacenar documentos JSON de prueba o datos semiestructurados con campos variables, ejecutar los comandos CRUD y consultas documentales	Permite explorar un esquema flexible y documentar la comparación SQL-NoSQL solicitada
Integración	Definir que datos se copian y en que momento, y con que identificador de origen, registrar el resultado de cada transferencia	Evita duplicidad sin control y facilita rastrear el registro desde la fuente original

4.4 Pruebas con las operaciones del CRUD

InsertMany y find en atenciones médicas:

```
db.Atenciones.insertMany([
    {
    _id: "EP_PRUEBA_ATENCION_001",
    ANHO: 2026,
    MES: 1,
    UBIGEO: "240201",
```

```
    DEPARTAMENTO: "TUMBES",
    PROVINCIA: "CONTRALMIRANTE VILLAR",
    DISTRITO: "ZORRITOS",
    SECTOR: "GOBIERNO REGIONAL",
    CATEGORIA: "I-4",
    CO_IPRESS: "1900",
    RAZON_SOC: "ZORRITOS",
    TIPO_SEXO: "2",
    GRUPO_EDAD: "2",
    ATENC_MEDICAS: "NE_00001",
  ATENC_NO_MEDICAS: 273,
  PERSONAS_ATENDIDAS: 116,
    },

    {
    _id: "EP_PRUEBA_ATENCION_002",
    ANHO: 2026,
    MES: 1,
    UBIGEO: "240201",
    DEPARTAMENTO: "TUMBES",
    PROVINCIA: "CONTRALMIRANTE VILLAR",
    DISTRITO: "ZORRITOS",
    SECTOR: "GOBIERNO REGIONAL",
    CATEGORIA: "I-4",
```

```
    CO_IPRESS: "1900",
    RAZON_SOC: "ZORRITOS",
    TIPO_SEXO: "1",
    GRUPO_EDAD: "6",
    ATENC_MEDICAS: 3,
  ATENC_NO_MEDICAS: "NE_00002",
  PERSONAS_ATENDIDAS: 8,
    }
])
```



db.Atenciones.find({_id:{$in:['EP_PRUEBA_ATENCION_001','EP_PRUEBA_ATENCION_002']}})

```
> db.Atenciones.find({_id:{$in:['EP_PRUEBA_ATENCION_001','EP_PRUEBA_ATENCION_002']}})
< {
    _id: 'EP_PRUEBA_ATENCION_001',
    ANHO: 2026,
    MES: 1,
    UBIGEO: '240201',
    DEPARTAMENTO: 'TUMBES',
    PROVINCIA: 'CONTRALMIRANTE VILLAR',
    DISTRITO: 'ZORRITOS',
    SECTOR: 'GOBIERNO REGIONAL',
    CATEGORIA: 'I-4',
    CO_IPRESS: '1900',
    RAZON_SOC: 'ZORRITOS',
    TIPO_SEXO: '2',
    GRUPO_EDAD: '2',
    ATENC_MEDICAS: 'NE_00001',
    ATENC_NO_MEDICAS: 273,
    PERSONAS_ATENDIDAS: 116
  }
```

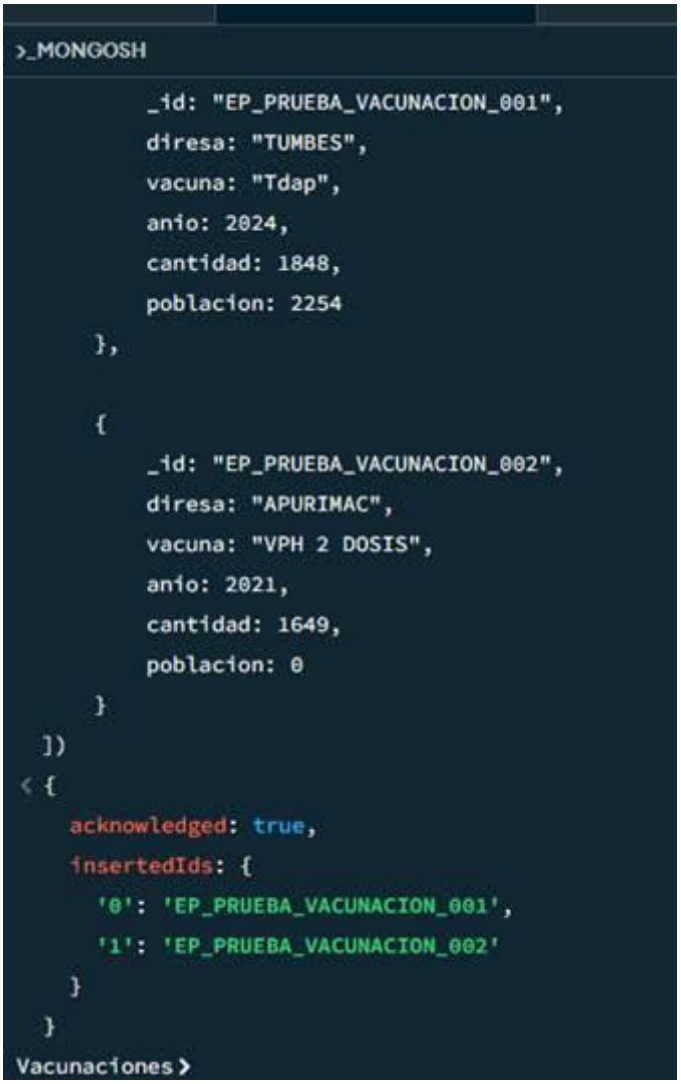
```
{
  _id: 'EP_PRUEBA_ATENCION_002',
  ANHO: 2026,
  MES: 1,
  UBIGEO: '240201',
  DEPARTAMENTO: 'TUMBES',
  PROVINCIA: 'CONTRALMIRANTE VILLAR',
  DISTRITO: 'ZORRITOS',
  SECTOR: 'GOBIERNO REGIONAL',
  CATEGORIA: 'I-4',
  CO_IPRESS: '1900',
  RAZON_SOC: 'ZORRITOS',
  TIPO_SEXO: '1',
  GRUPO_EDAD: '6',
  ATENC_MEDICAS: 3,
  ATENC_NO_MEDICAS: 'NE_00002',
  PERSONAS_ATENDIDAS: 8
}
```

Se inserto documentos en la colección de atención para ello se utilizó el comando de insertMany() ya que en este caso se insertó varios documentos en una sola operación. Mientras que el find nos sirve para consultar los valores insertados al documento.

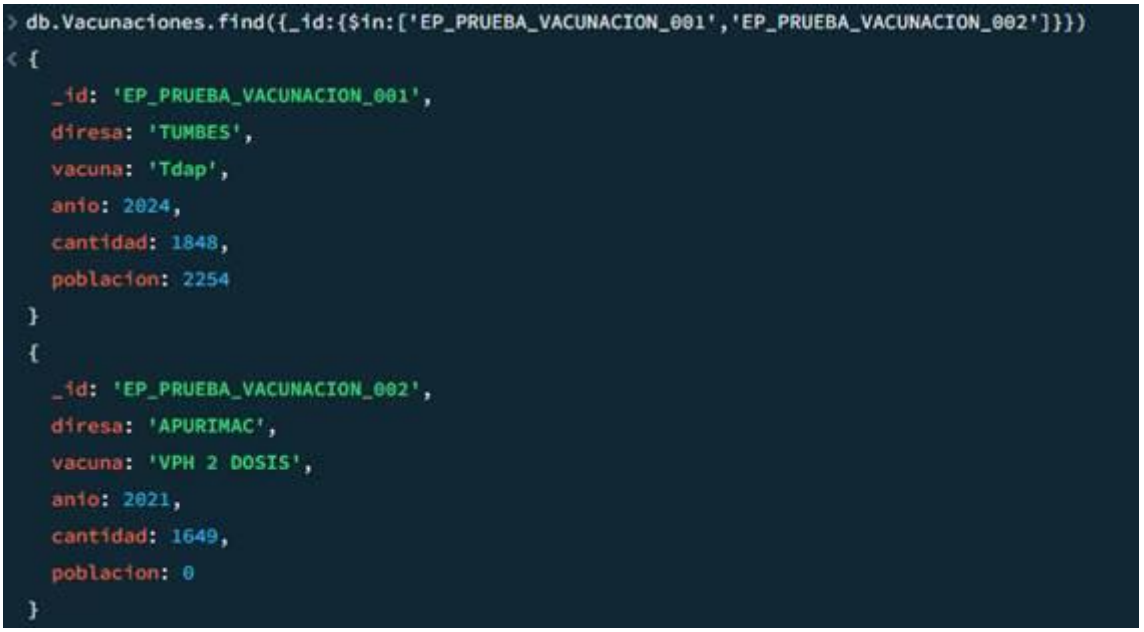
InsertMany y find en vacunaciones:

```
db.Vacunaciones.insertMany([
        {
        _id: "EP_PRUEBA_VACUNACION_001",
        diresa: "TUMBES",
        vacuna: "Tdap",
        anio: 2024,
        cantidad: 1848,
        poblacion: 2254
        },

        {
        _id: "EP_PRUEBA_VACUNACION_002",
        diresa: "APURIMAC",
        vacuna: "VPH 2 DOSIS",
        anio: 2021,
        cantidad: 1649,
        poblacion: 0
        }
])
```



db.Vacunaciones.find({_id:{$in:['EP_PRUEBA_VACUNACION_001','EP_PRUEBA_VACUNACION_002']}})



Se inserto documentos en la colección de vacunación para ello se utilizó el comando de insertMany() ya que en este caso se insertó varios documentos en una sola operación. En MongoDB se puede observar que conserva el cero en el apartado de poblacion como un valor numérico y no lo transforma en un valor NULL como pasaba en SQL Server. Mientras que el find nos sirve para consultar los valores insertados al documento.

Update en atención medica:

```
db.Atenciones.updateOne(
        {
        _id: "EP_PRUEBA_ATENCION_001"
        },
        {
        $set: {
         estado_revision: "REVISADO",
        }
        }
)
```

```
> db.Atenciones.updateOne(
    {
        _id: "EP_PRUEBA_ATENCION_001"
    },
    {
        $set: {
            estado_revision: "REVISADO",
        }
    }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
ANNO: 2026
MES: 1
UBIGEO: "240201"
DEPARTAMENTO: "TUMBES"
PROVINCIA: "CONTRALMIRANTE VILLAR"
DISTRITO: "ZORRITOS"
SECTOR: "GOBIERNO REGIONAL"
CATEGORIA: "I-4"
CO_IPRESS: "1900"
RAZON_SOC: "ZORRITOS"
TIPO_SEXO: "2"
GRUPO_EDAD: "2"
ATENC_MEDICAS: "NE_00001"
ATENC_NO_MEDICAS: 273
PERSONAS_ATENDIDAS: 116
estado_revision: "REVISADO"
```

Se actualiza el documento con el updateOne() para luego el comando $set permitirá modificar o agregar mas campos al documento y en los resultados que aparecen el matchedCount indica cuantos documentos coincidieron mientras que el modifiedCount indica cuantos documentos fueron modificados.

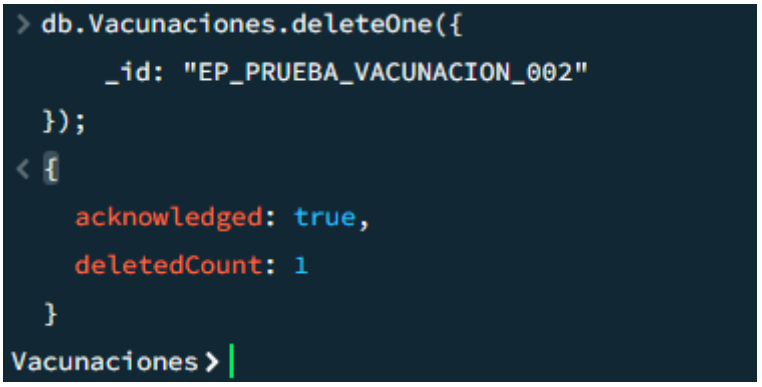
Siendo que el código funciona con éxito ya que se puede apreciar el estado de revisión a “REVISADO”.

deleteOne en vacunaciones:

db.Vacunaciones.deleteOne({

_id: "EP_PRUEBA_VACUNACION_002"

})



El deleteOne elimina un documento que coincide con el identificador y el deletedCount muestra cuantos documentos fueron eliminados por ende el documenta ha sido eliminado correctamente con el código de deleted.

4.5 Veredicto de visibilidad y ajuste de la ruta

La utilización de MongoDB es técnicamente viable para el desarrollo de la prueba de concepto, debido a que permite almacenar y gestionar los registros de atenciones y vacunaciones mediante documentos. Durante las pruebas realizadas, se comprobó el funcionamiento correcto de las cuatro operaciones CRUD: Insert, find, update y delete.

La operación Insert permitió registrar nuevos documentos en las colecciones, find nos permite consultar y recuperar los registros almacenados, update nos permite modificar la información de los documentos existentes y el delete permite eliminar los documentos seleccionados para la prueba. Los resultados que se obtuvieron demostraron que MongoDB puede gestionar correctamente la información de las bases de datos.

Por ende, se considera que MongoDB es viable para el alcance de la prueba ya que las operaciones de inserción consultan, actualización y eliminación se ejecutaron satisfactoriamente sin presentar errores durante la prueba.

Sobre el ajuste de la ruta del proyecto se incorpora la validación satisfactoria de las operaciones CRUD en MongoDB después de importar la base de datos. Al comprobarse el funcionamiento de las operaciones principales. Asimismo, se completó la comparación entre SQL Server y MongoDB, evaluando sus características y el manejo de los datos de atenciones y vacunaciones. Con los resultados obtenidos, se cuenta con información suficiente para identificar las ventajas de cada gestor y determinar su adecuación a los requerimientos del proyecto.